

Tile Movement Simulation (1-D)

Problem Statement

Given start and target board strings (e.g., 'BBB_RRR' -> 'RRR_BBB'), move tiles under rules: B moves right, R moves left, into '_' or jump over one tile into '_' to reach target in min moves.

Variants

- Return the sequence of boards

- Multi

How to Leverage the Solution

Use BFS on board states; adapt the neighbor generator for rule changes.

Java Solution

Below is a concise reference implementation. Full comments omitted for brevity.

```
import java.util.*;

public final class TileSwapSolver {
    public static int minMoves(String start, String target) {
        if (start.equals(target)) return 0;
        Queue<String> queue = new ArrayDeque<>(); Map<String, Integer> dist = new HashMap<>();
        queue.add(start); dist.put(start, 0);
        while (!queue.isEmpty()) {
            String cur = queue.poll(); int d = dist.get(cur);
            for (String nxt : nextStates(cur)) {
                if (dist.containsKey(nxt)) continue; if (nxt.equals(target)) return d + 1;
                dist.put(nxt, d + 1); queue.add(nxt);
            }
        }
        return -1;
    }

    private static List<String> nextStates(String s) {
        List<String> states = new ArrayList<>(); char[] arr = s.toCharArray(); int n = arr.length; int blank = s.indexOf('_');
        if (blank + 1 < n && arr[blank + 1] == 'R') states.add(swap(arr, blank, blank + 1));
        if (blank + 2 < n && arr[blank + 2] == 'R') states.add(swap(arr, blank, blank + 2));
        if (blank - 1 >= 0 && arr[blank - 1] == 'B') states.add(swap(arr, blank, blank - 1));
        if (blank - 2 >= 0 && arr[blank - 2] == 'B') states.add(swap(arr, blank, blank - 2));
        return states;
    }

    private static String swap(char[] arr, int i, int j) { char[] copy = arr.clone(); char t = copy[i];
    copy[i] = copy[j]; copy[j] = t; return new String(copy); }
}
```